

Manual Técnico — MercaducaOnLine

Resumen

- Proyecto: MercaducaOnLine
- Propósito: Plataforma web para abordar la problemática relacionada con el manejo, organización y acceso a la información sobre eventos, productos y emprendimientos dentro del local de emprendedores de Mercaduca.
- Stack: Frontend React (Vite en dev), Backend Node.js + Express, DB: Microsoft SQL Server (mssql).

Requisitos

- Node.js >= 18 recomendado
- npm >= 8
- SQL Server
- Puertos:
 - Backend: 5050 (por defecto)
 - Frontend dev (Vite): 5173 (por defecto)

Estructura relevante

- frontend/
 - package.json (scripts: dev, preview, start, build → vite build aplicado)
 - src/, public/
- backend/
 - server.js — monta rutas, sirve imágenes estáticas y el build del frontend
 - router/router.js — define las rutas de la API
 - controllers/ — controladores usados por las rutas:
 - controllers/Productos.js
 - controllers/Categorias.js
 - controllers/Puntaje.js
 - Imagenes/ — carpeta pública de imágenes

DB

- driver: mssql (Microsoft SQL Server)
- credenciales:

- user: "sa"
- password: "123456"
- server: "localhost"
- database: "db_mercaduca"
- port: 1433
- options:
 - encrypt: false
 - trustServerCertificate: true

Endpoints

- GET /productosMasVendidos
 - Devuelve productos con joins (Producto, EmprendimientoxProducto, Imagen, ProductosMasVendidos, Categoria)
- GET /Imagenes/:archivo
 - Archivos estáticos servidos desde backend/Imagenes
- GET * (catch-all)
 - Devuelve frontend (../frontend/dist/index.html)

Frontend: detalles y scripts

- Dependencias principales: react 19.x, react-dom, react-router-dom, axios
- Dev: vite (script "dev": "vite")
- Producción: el backend sirve ../frontend/dist; por tanto usar vite build para generar dist.

Backend: detalles y scripts

- Dependencias principales: express, cors, mssql
- Dev helper: nodemon (script "start": "nodemon server.js")
- Rutas definidas por ./router/router.js — abrir para documentar cada endpoint (métodos, body, respuestas, auth)

Instalación y ejecución (local)

1. Instalar dependencias
 - Frontend: cd /MercaducaOnLine/frontend npm install
 - Backend: cd /MercaducaOnLine/backend npm install
2. Desarrollo (frontend con Vite + backend)
 - En un terminal (backend): cd ../backend npm start

```
C:\Users\Sandra Garcia\Desktop\MercaducaOnLine\backend>node server.js
Backend corriendo en http://localhost:5050
```

Imagen 1: Backend inicializado

- En otro terminal (frontend): cd ../frontend npm run dev

```
C:\Users\Sandra Garcia\Desktop\MercaducaOnLine>cd frontend

VITE v7.1.12 ready in 2855 ms

➔ Local:   http://localhost:5173/
➔ Network: use --host to expose
➔ press h + enter to show help
```

Imagen 2: Frontend inicializado

- Abrir navegador al URL dev que indique Vite (por defecto <http://localhost:5173>)

3. Producción local (backend sirve build)

- Frontend: cd ../frontend npm run build # genera ../frontend/dist
- Backend: cd ../backend npm start
- Abrir: <http://localhost:5050>

1. GET /productos

- Descripción: devuelve listado general de productos (todos).
- Query: opcional (según implementación interna).
- Ejemplo curl: curl <http://localhost:5050/productos>
- Respuesta esperada (ejemplo): [{ "id_empxprod": 1, "nombre_producto": "Alfajor", "precio": 150, "codigo_imagen": "alfajor.jpg", "PuntajeProm": 4.5, "votos": 12, "categoria": "Dulces", "nombre_emprendimiento": "Dulces Mía", "descripcion": "Alfajor artesanal" }, ...]

2. GET /categorias

- Descripción: lista categorías disponibles.
- Ejemplo curl: curl <http://localhost:5050/categorias>
- Respuesta esperada (ejemplo): [{ "id": 1, "nombre": "Todos" }, { "id": 2, "nombre": "Dulces" }, ...]

3. GET /productos/categoria/:categoria

- Descripción: productos filtrados por nombre de categoría.
- Parámetros:
 - :categoria — nombre de la categoría (string)
- Ejemplo: curl <http://localhost:5050/productos/categoria/Dulces>
- Respuesta: misma estructura que /productos, sólo con productos de la categoría.

4. GET /productos/masvendidos/:categoria

- Descripción: productos más vendidos dentro de una categoría.
- Parámetros:
 - :categoria — nombre de la categoría
- Ejemplo: curl <http://localhost:5050/productos/masvendidos/Dulces>
- Respuesta: array de productos con métricas de ventas (depende de la consulta SQL en server.js / controlador).

5. POST /actualizarPromedio

- Descripción: recibe un voto y actualiza promedios en DB (producto y emprendimiento).
- Body (JSON) esperado (ejemplo): { "id_empxprod": 123, "voto": 5 }
- Ejemplo curl: curl -X POST <http://localhost:5050/actualizarPromedio>
-H "Content-Type: application/json"
-d '{"id_empxprod":123,"voto":5}'
- Respuesta esperada (ejemplo): { "promedioProducto": 4.6,
"promedioEmprendimiento": 4.2, "totalVotosProducto": 13 }

Rutas estáticas

- GET /Imagenes/:archivo — archivos estáticos servidos desde backend/Imagenes.
 - URL ejemplo: <http://localhost:5050/Imagenes/alfajor.jpg>

Controladores

- backend/controllers/Productos.js
 - Exporta: Productos, ProductosCategoria
 - Funcionalidad: consultas SQL para listar productos y por categoría.
- backend/controllers/Categorias.js
 - Exporta: Categorias, MasVendidoPorCategoria
- backend/controllers/Puntaje.js
 - Exporta: ActualizarPromedio
 -

Base de datos

- Motor: Microsoft SQL Server (mssql)
- Tablas:
 - Producto (id, nombre_producto, descripcion, precio, id_categoria, ...)
 - EmprendimientoxProducto (id_empxprod, id_producto, id_emprendimiento, id_imagen, ...)
 - Imagen (id, codigo_imagen, ruta, ...)
 - Categoria (id, nombre)
 - ProductosMasVendidos (id_producto, cantidad_vendida, ...)
 - Puntajes/Votos (id, id_empxprod, id_usuario?, valor, fecha)
 - Emprendimiento (id, nombre_emprendimiento, promedio_emprendimiento, ...)

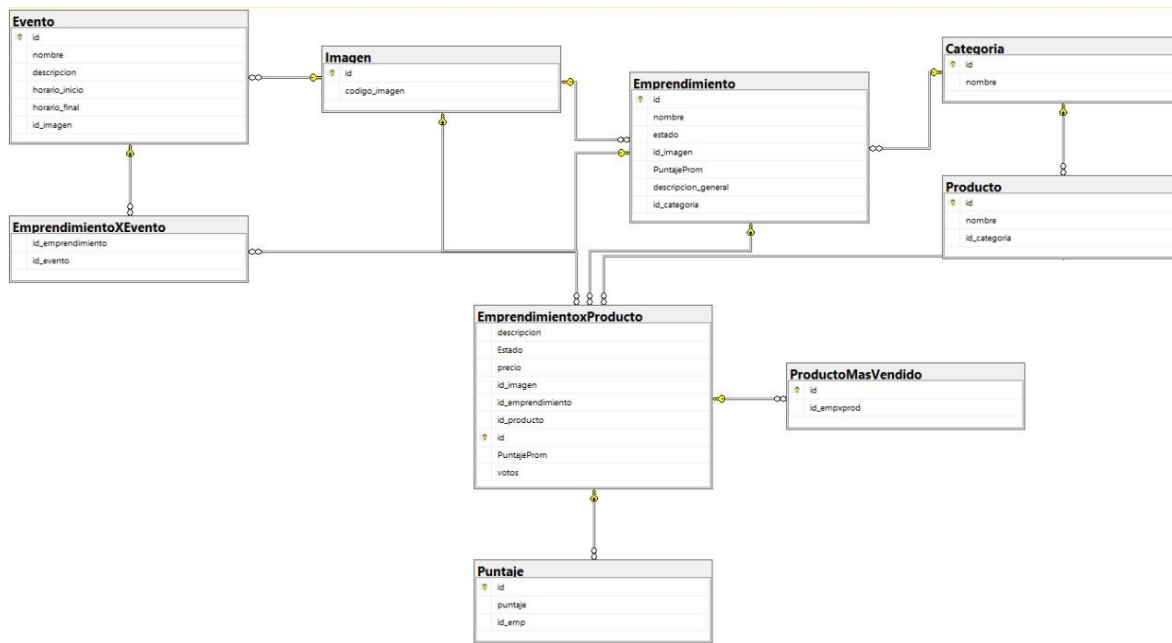


Imagen 3: Diagrama relacional de la base de datos

Despliegue sugerido

- Frontend: build con `npm run build` (vite build) → `/frontend/dist`
- Backend: Dockerfile + docker-compose que incluya SQL Server (si se usa localmente) o conectar DB gestionada.
- Servir backend detrás de nginx con SSL y reverse proxy al puerto de Node.

Comprobaciones y comandos útiles

- Ver puertos: `ss -ltnp | grep -E ':5050|:5173|:3000'`
- Probar API: `curl http://localhost:5050/categorias`
- Construir y servir producción: `cd frontend && npm run build cd backend && npm start xdg-open http://localhost:5050`

Historia del documento

- v0.3 — 26-11-2025